

Document Set Overview

This package consolidates the finalized architecture into four working documents needed to begin build: a Product Requirements Document (PRD), a Technical Design Document (TDD), a System Architecture reference, and a Design System guide. All technical decisions reflect the locked hybrid architecture — Spring Boot for core product logic, FastAPI for AI orchestration, on-device inference for latency-critical tasks, and free-tier cloud APIs for heavy AI compute.

1. Product Requirements Document (PRD)

1.1 Product Vision

Quilore is a cross-platform fitness and nutrition app that replaces a human trainer's core functions — form correction, load recommendation, program design, macro tracking, and diet feedback — using voice interaction, computer vision, and conversational AI, with specific accuracy for Indian/South Asian food and regional exercise needs.

1.2 Target User

Fitness-focused individuals, likely intermediate-to-advanced lifters, who want data-driven training feedback without a paid human coach, and who eat a predominantly Indian/regional diet that generic calorie apps misclassify.

1.3 Core Feature Set

Fitness Module

Feature	Description	Priority
Voice-driven logging	Log sets/ reps/ weight hands-free via speech, parsed on-device and structured via cloud LLM	P0
Workout import	Import routines from pasted text, photo (OCR), or video (frame + pose parsing)	P0
Animated exercise demos	Show correct-form animations per exercise with targeted muscle overlays	P0
AI form-check	Analyze submitted lift video for joint angles, bar path, and rep velocity to flag poor form	P0
Ego-lifting / load detection	Flag excessive load based on velocity loss and form breakdown; recommend weight up/down	P0
Superset & workout editing	Let user request supersets or modify any planned workout via chat	P0
Personalized programming	AI researches and generates a tailored program based on goals and history	P1
Muscle-map injury triage	Tap a muscle region, describe pain, get a fatigue-vs-injury risk flag (not diagnostic)	P0

Feature	Description	Priority
Conversational AI coach	Chat interface for all workout questions, modifications, and guidance	P0

Nutrition Module

Feature	Description	Priority
Food photo capture	Upload/take photo of plate or bowl	P0
Regional dish recognition	Identify Indian/regional dishes accurately (e.g., dalma vs. generic dal)	P0
User confirmation loop	Confirm/correct detected dish list before calculation	P0
Portion/calorie estimation	Estimate grams and calories from image, shown as a range with manual adjustment	P0
Manual meal entry with portion-based macro calculation	Let users type foods and portion sizes manually; the app calculates calories, protein, carbs, and fat automatically from the nutrition database	P0
Goal-based calorie and macro targets	Calculate daily calorie and macro targets based on goal, weight, height, and related profile inputs; let users track progress against those targets	P0
Food-quality feedback	Flag excess oil/fat visually; offer improved recipe on request	P1
Combination & timing analysis	Flag poor food pairings and eating-time patterns	P1
Cross-module insight	Correlate nutrition patterns with training performance in coaching responses	P1

1.4 Non-Goals (Explicitly Out of Scope for v1)

- Medical diagnosis of injuries (the app only flags risk, never diagnoses).
- Fully automatic, error-free portion estimation — outputs are always ranges with manual override for scanned meals and remain editable for manually entered meals.
- Support for cuisines beyond Indian/South Asian in the first release (architecture is extensible, but v1 nutrition accuracy is scoped to this region).

1.5 Success Metrics

- Voice logging transcription accuracy in gym-noise conditions.
- Form-check flagging precision against user-reported/coach-reviewed lifts.
- Dish recognition accuracy on regional Indian dishes (measured against the Khana Hard-20 subset benchmark).
- Percentage of nutrition estimates requiring manual correction (target: decreasing over time as the synonym-mapping table improves).
- Percentage of manual meal entries resolved to valid macro totals without user abandoning the flow.
- Percentage of active users who set a calorie or macro goal and log against it weekly.

2. Technical Design Document (TDD)

2.1 Service Ownership Matrix

Domain	Owner	Technology
Auth, user profiles, workout logs, nutrition logs, meal entries, goal targets, sync, plans, progression rules, quotas	Core App Backend	Java Spring Boot
AI gateway, provider routing, OCR cleanup, prompt assembly, embeddings, food-image orchestration	AI Orchestration	Python FastAPI
Unified RAG storage, user data, workout/nutrition logs, media metadata	Data Storage	PostgreSQL + pgvector + MinIO
Client UI, camera/mic integration, exercise animation rendering	Mobile Client	React Native

2.2 Architectural Boundary Rule

FastAPI handles stateless AI processing and formatting; Spring Boot enforces all stateful product logic. This guarantees AI cannot autonomously bypass user progression rules or database integrity without passing through business validation in Spring Boot.

2.3 AI Inference Distribution

Edge (On-Device)

- **Voice:** Whisper.cpp handles continuous voice recognition on-device to mitigate gym noise and network latency.
- **Vision/Pose:** MediaPipe Pose processes real-time skeletal tracking for form and ego-lifting detection directly on the device, avoiding server-side GPU costs.
- **OCR:** Initial OCR for workout imports runs on-device first; cloud vision models are reserved strictly as a fallback for complex or degraded images that fail on-device processing.

Cloud API (Serverless, Free-Tier)

- Heavy LLM and advanced vision tasks route to free-tier serverless providers — Groq, OpenRouter, Hugging Face Inference Providers, and NVIDIA NIM — eliminating the need for self-hosted GPU infrastructure.
- The FastAPI gateway maintains a **priority routing / fallback matrix**: primary provider (e.g., Groq for LLM tasks, due to fastest free-tier inference) with automatic fallback to OpenRouter or Hugging Face on rate limit (HTTP 429), timeout, or degradation.
- **Background queuing:** Non-urgent AI tasks (e.g., generating a full personalized program, batch nutrition-pattern summaries) route through a quota-aware background queue so they don't compete with real-time chat/voice requests for limited free-tier quota.

2.4 Resilience Pattern

FastAPI implements retry, backoff, and circuit-breaker logic (via a library such as Tenacity) for all external AI calls:

- **Exponential backoff with jitter:** Failed calls to a rate-limited provider pause with increasing, randomized delay before retry.
- **Async circuit breaker:** Repeated failures past a threshold trip the circuit to an OPEN state, failing fast rather than holding dead connections.
- **Graceful degradation:** On circuit-open, a deterministic fallback (e.g., "Your workout has been saved; analysis will follow shortly") returns instead of an error, so the client never sees a crash.

2.5 Data Model (RAG Layer)

Column	Type	Purpose
id	UUID/SERIAL	Primary key
content	TEXT	Raw text chunk (workout log, chat turn, IFCT nutrient profile)
metadata	JSONB	User ID, timestamps, document type — used for exact filtering
embedding	vector(768)	Semantic embedding for similarity search

Text is chunked at 200–500 tokens with 50–100 token overlap before embedding, to preserve semantic context across boundaries. Retrieval uses HNSW indexing for the static exercise/nutrition corpus (higher recall) and IVFFlat for larger, more dynamic user-log partitions, with cosine similarity as the primary distance metric, combined with keyword/JSONB filtering in a hybrid search pattern to reduce hallucination.

2.6 Nutrition Data Extensions

Entity	Key Fields	Purpose
meal_entry	id, user_id, source_type, meal_type, timestamp, notes	Stores each meal log whether scanned or manually entered
meal_item	id, meal_entry_id, food_name, source_confidence, portion_value, portion_unit, grams, calories, protein_g, carbs_g, fat_g	Stores food-line items and computed macros
nutrition_goal	id, user_id, goal_type, weight_kg, height_cm, age, sex, activity_level, target_calories, target_protein_g, target_carbs_g, target_fat_g, effective_from	Stores daily targets derived from user profile and selected goal
nutrition_progress_snapshot	id, user_id, date, consumed_calories, consumed_protein_g, consumed_carbs_g, consumed_fat_g, target_calories, target_protein_g, target_carbs_g, target_fat_g	Stores daily rollups for progress views and analytics

2.7 Offline-First Sync

The mobile client uses WatermelonDB (SQLite-backed) for local caching of exercises, logs, chat history, meal entries, goal targets, and daily nutrition progress. Sync follows a deterministic pull-push protocol: the client sends a `last_pulled_at` timestamp, the backend returns a delta payload, and local mutations are pushed as an atomic transaction, with conflict resolution via last-write-wins timestamps (or CRDTs for cumulative metrics like total training volume and daily intake tallies).

2.8 Form-Check Kinematic Method

MediaPipe Pose extracts 33 three-dimensional landmarks per frame. Joint angles are derived via dot products between vectors from a central joint to adjacent landmarks; postural deviations (e.g., spinal rounding) use vector cross products against a global up-vector. Dynamic Time Warping normalizes rep duration against reference templates, and Normalized Cross-Correlation scores bar-path similarity; a sharp drop in concentric velocity combined with form deviation triggers the ego-lifting heuristic and downward load recommendation.

2.9 Nutrition Pipeline Method

Scanned Meal Flow

1. Image capture → segmentation (SAM 2, hosted via cloud inference) isolates each food item and its container.
2. Depth estimation (Depth Anything V2, hosted via cloud inference) reconstructs relative 3D surface geometry.
3. Classification (ViT-based model, cloud-hosted) identifies dishes; ViT is selected over CNN baselines due to substantially higher accuracy on the Khana "Hard-20" subset.
4. Regional-name resolution maps recognized classes to the closest IFCT 2017 entry via a maintained synonym table (e.g., *dalma* → *dal* category), expanded through user corrections.
5. User confirms/edits the candidate list via chat UI before calculation.
6. Scale calibration (reference object or saved plate diameter) converts relative depth to absolute volume, then a food-class density factor converts volume to grams.
7. Nutrient values are queried from IFCT 2017; results are shown as a range with a manual gram-adjustment slider, since single-image volume estimation is an inherently ill-posed scale problem.
8. Quality reasoning (oil/fat visual cues, combination checks, meal timing) runs via the LLM, grounded in the confirmed dish's nutrient profile.

Manual Meal Flow

1. User enters one or more foods and portion sizes using natural language or structured fields.
2. Spring Boot normalizes units (e.g., roti count, cups, grams, bowls, tablespoons) into canonical serving representations.

3. FastAPI may optionally help parse ambiguous free-text meal descriptions into structured food items when needed.
4. Food names are resolved to IFCT 2017 entries through exact match, synonym mapping, and curated aliases for Indian/regional foods.
5. The backend calculates calories, protein, carbs, and fat automatically for each line item and aggregates meal totals.
6. Users can edit food matches, serving units, and grams before final save.

2.10 Goal-Based Macro Target Engine

1. User enters goal, weight, height, and related profile inputs such as age, sex, and activity level.
2. Spring Boot calculates maintenance calories and applies a configurable surplus or deficit policy based on the selected goal.
3. Macro targets are derived from calorie targets using configurable protein, fat, and carb allocation rules.
4. Targets are versioned so historic meal logs remain tied to the goal state active on that date.
5. Daily progress services aggregate consumed vs. target calories and macros for charts, streaks, and coaching prompts.

3. System Architecture

3.1 High-Level Topology

The architecture separates four layers: client (React Native), backend (Spring Boot + FastAPI), data (PostgreSQL/pgvector + MinIO), and external services (cloud AI providers, wger, IFCT 2017). The mobile client talks to Spring Boot directly for auth, logs, sync, nutrition targets, manual meal entry persistence, and progress tracking, and to FastAPI for anything requiring AI processing; FastAPI in turn hands validated, structured results back to Spring Boot for persistence, ensuring AI never writes directly to the source-of-truth database without business-rule validation.

3.2 Voice Logging Data Flow

Voice input is transcribed on-device via Whisper.cpp, sent to the FastAPI gateway for structuring into JSON via a cloud LLM, then validated and persisted through Spring Boot.

3.3 Nutrition Logging Data Flow

Scanned food photos are processed through segmentation, classification, and depth estimation on cloud APIs, presented to the user for confirmation, converted to a nutrient estimate against IFCT 2017, and enriched with LLM-generated quality feedback before being persisted as a log entry. Manual meal entries bypass the vision stack and go through Spring Boot nutrition resolution services to calculate calories and macros directly from food names and portion sizes.

3.4 Goal Tracking Data Flow

The user sets a nutrition goal by entering objective, body metrics, and related profile inputs. Spring Boot computes target calories and macro splits, stores the effective target set, and updates daily progress snapshots as meals are logged through either scanning or manual entry. The coaching layer can then use these snapshots to answer questions such as whether the user is under protein, over calories, or aligned with the selected goal.

3.5 External Data Dependencies

Source	Role	Notes
wger REST API	Exercise taxonomy, equipment, muscle-group SVGs	Self-hosted instance recommended for control and offline caching
IFCT 2017	Indian nutrient composition database (528 foods)	Published by NIN/ICMR; used as authoritative nutrient source
Khana dataset	Indian food image classification training/benchmark data	131K images, 80 labels, used to validate/fine-tune vision models
IndianFoodNet	Supplementary Indian dish image dataset	30 dish classes, ~5,500 images

3.6 Provider Fallback Matrix (AI Tasks)

Task	Primary	Fallback 1	Fallback 2
Chat/LLM reasoning	Groq (Llama 3.3/4)	OpenRouter free pool	Hugging Face Inference Providers
Speech-to-text	On-device Whisper.cpp	— (no cloud fallback needed)	—
Food vision classification	Hugging Face Inference Providers / Gemini free tier	NVIDIA NIM	OpenRouter multimodal
Segmentation (SAM 2)	Hugging Face / Cloudflare Workers AI	—	—
Depth estimation (DA-V2)	Hugging Face	—	—
Embeddings	Cloud embedding API (e.g., Gemini text-embedding-004)	Hugging Face free embedding endpoint	—
Free-text meal parsing	Groq	OpenRouter free pool	Spring Boot rule-based parser

This table operationalizes the "priority routing matrix" requirement from the locked architecture spec, giving explicit primary/fallback order per task.

4. Design System

4.1 Navigation Structure

Tab-based navigation across six primary sections: Workout, Chat, Progress, Muscle Map, Nutrition, Profile — chosen for cognitive simplicity during physical exertion.

4.2 Core UI Patterns

Pattern	Used In	Behavior
Confirmation chat card	Workout import, food recognition	Shows AI-detected items as an editable list; user taps to confirm/correct before data is persisted
Range-based estimate display	Calorie/portion results for scanned meals	Never shows a single false-precision number; always a range plus a manual slider
Macro target progress card	Nutrition home, daily summary, profile goal setup	Shows consumed vs. target calories, protein, carbs, and fat with remaining balance and visual progress bars
Manual meal composer	Nutrition logging	Lets users add foods, select portion units, and review auto-calculated calories and macros before save
Interactive muscle map (SVG)	Injury triage, exercise targeting	Tappable front/back body diagram; tap opens a pain-descriptor form
Voice capture indicator	Workout logging	Persistent mic button with live partial-transcript overlay during recording
Animated form demo	Exercise detail view	Looping reference animation with overlaid targeted-muscle highlight

4.3 Visual Language Principles

- High-contrast, large-touch-target UI for use mid-workout (sweaty hands, quick glances).
- Muscle-map and exercise-demo visuals reuse wger's existing SVG/anatomical assets to avoid custom asset production for v1.
- Nutrition confirmation screens use a conversational, low-friction tone ("I see dalma, rice, and bhindi sabzi — is this right?") rather than a rigid form, reducing correction fatigue during regional dish onboarding.
- Manual meal entry should feel faster than spreadsheet-style calorie apps by supporting common Indian household units such as roti, katori, bowl, cup, spoon, and piece.
- Feedback from the AI coach (form corrections, nutrition flags) is visually distinguished from raw data logs, so users can tell "AI observation" apart from "confirmed fact."

4.4 Accessibility & Trust Signals

- All AI-derived health/injury guidance is labeled as a risk-flag, not a diagnosis, directly in the UI copy, consistent with the architecture's framing of the triage engine.
- Every AI-detected value (food item, exercise, calorie count) is editable inline — no AI output is ever presented as final without a user-accessible correction path.

- Goal-based calorie and macro targets are labeled as coaching estimates, with inputs and formulas reviewable by the user so targets do not appear medically prescriptive.